
Systemes Embarqués



ROB 2A, UE 4.1, ENSTA

Développer avec Docker

SYSTÈMES EMBARQUÉS

Créer/Modifier une image

De petites modifications de l'image peuvent être effectuées en procédant comme suit :

- télécharger l'image à modifier (depuis Docker Hub par exemple)
- exécuter un conteneur avec cette image en mode interactif (option `-it`), cela devrait ouvrir un terminal dans le conteneur
- appliquer les modifications à l'aide des commandes du terminal
- quitter le terminal
- enregistrer les modifications dans une nouvelle image (avec un nom différent)

=> Problème de traçabilité des dépendances

Créer/Modifier une image

L'utilisation d'un Dockerfile est largement recommandée pour créer une image.

Les instructions du Dockerfile permettent de construire proprement la nouvelle image à partir de l'image de base.

Mais aussi, **le Dockerfile permet de tracer les dépendances et les modifications apportées à l'image de base**

```
1 FROM ros:humble
2
3 # Install necessary packages for seabot2-ros
4 RUN apt-get update -y
5 RUN apt-get install -y libi2c-dev
6 RUN apt-get install -y python3-pip
7 RUN apt-get install -y libboost-all-dev
8 RUN pip install Jinja2
9 RUN apt-get install -y libgps-dev
10 RUN apt-get install -y libproj-dev
11 RUN apt-get install -y libncurses-dev
12
13 # Install GeographicLib
14 RUN apt-get install -y wget unzip
15 RUN mkdir -p /home/LIBS \
16     && cd /home/LIBS \
17     && wget https://sourceforge.net/projects/geographiclib/files/distrib/2.5/GeographicLib-2.5.zip \
18     && unzip -q GeographicLib-2.5.zip \
19     && rm GeographicLib-2.5.zip \
20     && cd GeographicLib-2.5 \
21     && mkdir BUILD \
22     && cd BUILD \
23     && cmake .. \
24     && make \
25     && make install
```

Exemple de Dockerfile

Remarques

La plupart du temps, nous n'avons pas besoin de garder le conteneur après exécution (utilisez `-rm` dans `docker run`)

ATTENTION:

Les images et conteneurs prennent beaucoup d'espace, pensez à les supprimer si ils ne sont plus utiles avec `docker image rm`

Executing (running) a container

```
docker run -it --rm --name container_name image_name
```

- `-it` : interactive mode, open a `"/bin/bash"` terminal in the container

- `--rm` : remove the container at the end of execution (!

Commits on the modified image cannot be done as container will disappear when it's execution terminates)

- `--name container_name` : give a name to the container
- `image_name` : name of the image to execute in the container

Executing another command in a running container

```
docker exec -it container_name cmd
```

- `-it` : interactive mode
- `container_name` : name of the container
- `cmd` : command to execute in the container, e.g. `/bin/bash` to open a terminal in the container

Sharing folders

The `-v` option allows to share a folder (or a file) between the host computer (i.e. your computer) and the container.

Sharing display

- UNIX (Linux is a kind of Unix) uses X11 graphical user interface (GUI)
- The display is defined in the DISPLAY environment variable (can be shown with "echo \$DISPLAY")
- The display uses a folder "/tmp/.X11-unix/" with the sockets

than connects graphical clients to the graphical server.

- We just need to give the DISPLAY variable to the container and share "/tmp/.X11-unix/" folder :

`-e DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix`

- The host computer must allow the docker container to display graphical application, this permission is given by :

`xhost +local:docker`